Unlimited Practice Opportunities: Automated Generation of Comprehensive, Personalized Programming Tasks

Sven Jacobs
University of Siegen
Siegen, Germany
sven.jacobs@uni-siegen.de

Henning Peters
University of Siegen
Siegen, Germany
henning.peters@student.uni-siegen.de

Steffen Jaschke
University of Siegen
Siegen, Germany
steffen.jaschke@uni-siegen.de

Natalie Kiesler
Nuremberg Tech
Nuremberg, Germany
natalie.kiesler@th-nuernberg.de

Abstract

Generative artificial intelligence (GenAI) offers new possibilities for generating personalized programming exercises, addressing the need for individual practice. However, the task quality along with the student perspective on such generated tasks remains largely unexplored. Therefore, this paper introduces and evaluates a new feature of the so-called Tutor Kai for generating comprehensive programming tasks, including problem descriptions, code skeletons, unit tests, and model solutions. The presented system allows students to freely choose programming concepts and contextual themes for their tasks. To evaluate the system, we conducted a two-phase mixed-methods study comprising (1) an expert rating of 200 automatically generated programming tasks w.r.t. task quality, and (2) a study with 26 computer science students who solved and rated the personalized programming tasks. Results show that experts classified 89.5% of the generated tasks as functional and 92.5% as solvable. However, the system's rate for implementing all requested programming concepts decreased from 94% for single-concept tasks to 40% for tasks addressing three concepts. The student evaluation further revealed high satisfaction with the personalization. Students also reported perceived benefits for learning. The results imply that the new feature has the potential to offer students individual tasks aligned with their context and need for exercise. Tool developers, educators, and, above all, students can benefit from these insights and the system itself.

CCS Concepts

• **Social and professional topics** → **Computing education**; • **Computing methodologies** → **Artificial intelligence**.

Keywords

Programming Education, Programming Exercises, Context Personalization, Large Language Models, Generative AI

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.
ITiCSE 2025, Nijmegen, Netherlands
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1567-9/2025/06
https://doi.org/10.1145/3724363.3729089

ACM Reference Format:
Sven Jacobs, Henning Peters, Steffen Jaschke, and Natalie Kiesler. 2025. Unlimited Practice Opportunities: Automated Generation of Comprehensive, Personalized Programming Tasks. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2025), June 27-July 2, 2025, Nijmegen, Netherlands.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3724363.3729089

1 Introduction

Programming is a fundamental Computer Science (CS) education component. CS students usually learn how to program through hands-on practice and gaining experience. The design of programming tasks, however, is crucial. Research has shown that contextualizing tasks, i.e., connecting them to topics relevant to students' personal and cultural interests and experiences, can increase their situational interest [5, 22] and performance [36]. Creating high-quality programming tasks addressing interests of a diverse student body remains a challenge as educators' resources are limited.

Advances in Generative AI (GenAI) models and tools greatly impact computing education [4, 16, 27, 28]. GenAI models can provide automated, personalized feedback to student solutions through chatbots [3, 18, 30, 31] and other tools [11, 13, 17, 25]. This is potentially interesting for educators preparing feedback for learners and students' self-paced practice at home. The second relevant point refers to the capabilities of GenAI tools to automatically generate educational materials, such as programming tasks [15, 29]. These developments create an unprecedented opportunity: Offering students unlimited, personalized programming practice with immediate feedback, without increasing an instructor's workload. Prior research has shown promising systems for generating contextualized programming tasks [7, 20]. However, questions concerning task quality and the student perspective remain open.

It is the **goal** of this study (a) to introduce a new feature of the Tutor Kai for generating comprehensive programming tasks including problem descriptions, code skeletons, unit tests, and model solutions; (b) to gather students' and experts' perspectives on programming tasks that are personalized.

The **contributions** are as follows: (1) a feature for generating personalized programming tasks that provide all components required for automated assessment and feedback, (2) an expert rating of the quality of these generated components, and (3) insights into how students rate and use such a feature and tasks generated by it.

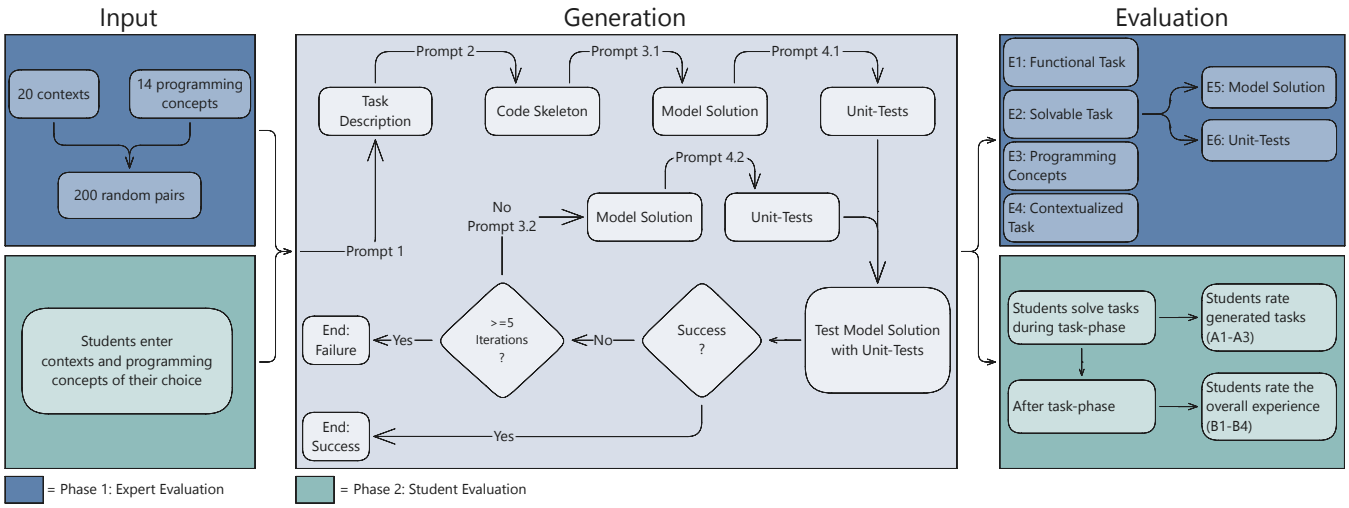

Figure 1: Overview of task generation pipeline and evaluation

2 Related Work

Recent developments in GenAI have led CS educators to adopt GenAI tools for creating teaching materials [26, 28]. Potential applications for various educational resources [8] have been explored, ranging from quiz questions [21] and debugging exercises [23] to tasks for both block-based [1], and visual programming [24].

In the context of programming exercises, Sarsa et al. [29], [9] demonstrated that Large Language Models (LLMs) like Codex can create thematically and conceptually relevant programming tasks, including problem descriptions, model solutions, and unit tests. Unfortunately, only 30% of their generated tasks had model solutions that passed the corresponding generated tests [29].

Speth et al. [34] conducted a case study using ChatGPT to create exercise sheets for a beginner programming course. While the generated exercises were well-received by students, they observed that most tasks required minor manual adjustments before deployment.

Del Carpio Gutierrez et al. [7] investigated contextually personalized programming tasks. Their expert evaluation found that GPT-4 could generate high-quality problem descriptions and model solutions [7]. By using GPT-3.5 to generate Parsons problems they showed that students valued the ability to customize both programming concepts and contexts while reporting perceived learning benefits [6]. Similarly, Logacheva et al. [20] investigated student interactions with contextually personalized programming tasks, finding high satisfaction with contextual personalization features. However, 20% of the exercises contained faulty code. They had prefiltered tasks and restricted students to choosing from predefined programming concepts and contexts [20].

The generation of unit tests based on problem descriptions and reference solutions was subject to research by Alkafaween et al. [2]. Their study shows that LLM-generated test suites can correctly identify most valid solutions. In terms of quality, they achieved similar comprehensiveness compared to instructor-created tests. Torres [35] developed CodeContrast, an approach that uses contrastive learning to generate coherent programming exercises by mapping problem descriptions, test cases and model solutions into a shared feature space.

In contrast to previous work, our approach focuses on generating comprehensive programming tasks with all components necessary for automated formative assessment and feedback. The use case is thus that students can immediately attempt to solve the generated exercise, e.g., by using the generated code skeleton, and receive unit test results and other feedback as needed. While earlier studies have explored individual aspects (i.e., test generation or task contextualization) no prior work has evaluated a system where *all* task components are generated on-demand based on student-entered concepts and contexts. In contrast to previous work [20], which attempted to control task difficulty through explicit prompting, we chose to vary the number of programming concepts integrated into a single task (which is more authentic). Additionally, we use a comprehensive generation pipeline to improve task quality, addressing the code quality issues reported in previous studies [20, 29, 34].

3 Task Generation Pipeline in the Tutor Kai

Our approach generates programming tasks with all components necessary for automated assessment and feedback as part of the Tutor Kai. The system enables students to freely enter programming concepts and contexts as a basis for task generation. It further allows students to solve the generated task immediately.

We developed and implemented a generation pipeline (see Figure 1: Generation) that creates complete programming tasks for students based on the selection of one or multiple programming concepts (e.g., recursion) and a context (e.g., music). It generates the following task components:

(1) *Task Description:* A textual description of a programming task that students need to solve. The description is contextualized. It serves as primary instruction for students.
(2) *Code Skeleton:* A basic structure providing the method signatures and necessary comments indicating where to implement a solution.

(3) *Unit Tests:* Automated test cases that verify student solutions by comparing expected and actual outputs.
(4) *Model Solution:* A fully implemented, executable solution that serves two purposes: (1) provide a reference for instructors when answering student questions, and (2) as an internal validation mechanism during the generation process to ensure the unit tests are correctly specified.

Each task component was generated by a separate prompt. Our approach includes multiple prompting techniques including role prompting, style prompting [32], few-shot examples and reflection [33]. The reflection component, for example, automatically evaluates generated unit tests and model solutions by executing the model solution against unit tests. If the evaluation fails, the system provides this feedback including compiler output and unit test results back to the GenAI system to improve both the unit tests and model solution. We limited the maximum number of improvement iterations to five after an initial piloting. We found that the results did not improve any further after five iterations. These internal iterations are hidden from students.

The separation of task components and their generation via individual prompts led to multiple benefits: (1) more specific and focused prompts, (2) reduced context noise by only including relevant information, and (3) flexibility to use different (potentially fine-tuned) models and parameters (e.g. temperature) specialized for each component. All prompts are available in our supplementary materials repository [12].

4 Methodology

The evaluation of the generated programming tasks is guided by the following research questions (RQs):

RQ1 *How do experts rate the quality of the personalized programming tasks?*
RQ2 *How do students use and rate personalized programming tasks?*

To answer these questions, we applied a two-phase mixed-methods approach. RQ1 focused on rating the task quality through experts, while RQ2 examined student usage and assessment of the generated tasks.

4.1 Phase 1: Expert and Automated Assessment

4.1.1 Data Collection. To systematically evaluate the task quality, we generated 200 Python programming tasks. 100 tasks required a single programming concept. The remaining 100 tasks addressed multiple concepts (50 of these tasks addressed two concepts, and 50 of them three concepts). We used the described pipeline to automatically generate all programming tasks (see Figure 1: Generation).

Table 1: Contexts and concepts used for task generation

Contexts: 20
Amusement Park; Animals; Aquarium; Basketball; Cooking; Film; Fishing; Gardening; Mental Health; Modern Gaming; Music; Olympics; Pets; Relationships; Restaurant; Rugby; Social Media; Sports; Streaming Services; Virtual Reality
Programming Concepts: 14
Boolean and None; Operations with numbers; String; Integer; Float; For loops; While loops; If-Else statements; Logical operators (==, !=, <, >, <=, >=, or, and, not); Functions as variables; Higher-order functions; Recursion; Lists; Tuples

The contexts (e.g., animals, basketball, cooking) in Table 1 were adapted from del Carpio Gutierrez et al. [6]. The 14 programming concepts were selected from Python lecture materials used in an introductory 'Object-Oriented and Functional Programming' (OOFP) course. For task generation, we randomly paired contexts with programming concepts. All 200 generated tasks are available in our supplementary materials repository [12].

4.1.2 Data Analysis. We conducted an expert assessment of all 200 generated tasks. The assessment criteria (E1-E6) were adapted from previous studies on automated programming task generation [7, 20, 29]. Experts were asked to provide a binary classification (criteria met/not met) for the categories E2-E6 (E1 was assessed automatically, see below).

E1 *Functional Task:* Model solution and unit tests are syntactically correct and executable. The generated model solution passes all generated unit tests, demonstrating there is at least one valid solution. This automated assessment rubric (cf. [29]) was performed automatically during task generation (see section 3).
E2 *Solvable Task:* The task description provides complete specifications for the required implementation. This is crucial as solutions are evaluated through automated unit testing, demanding that all information required to pass these tests is explicitly stated in the task description.
E3 *Programming Concepts:* The task incorporates all intended programming concepts, either through explicit requirements in the task description or through their application in the implementation.
E4 *Contextualized Task:* The generated task description matches the requested context. We consider a task contextually appropriate if its scenario is connected to the chosen context.
E5 *Model Solution:* The model solution correctly implements the task specifications. Tasks can fail this criterion even though they are functional (E1), e.g., if the unit tests checked cases not specified in the task description.
E6 *Unit tests:* The unit tests accurately reflect and test the explicit requirements in the task description, without introducing additional requirements or edge cases.

As a complete specification of requirements is necessary to assess the correctness of implementations and test cases, the evaluation of model solutions (E5) and unit tests (E6) was only conducted for solvable tasks (rated as E2). Specific issues that led to "No"-responses were documented. Furthermore, we tracked the number of successfully incorporated programming concepts per task, and the number of reflection iterations required to generate functional tasks. Moreover, we cross-checked the results w.r.t. the number of contained programming concepts.

4.2 Phase 2: Student Evaluation

4.2.1 Participants and Setting. The study was conducted mid-term in the OOFP tutorial. Usually, students seeking assistance with their homework attend, as it is voluntary. For this session, approximately 200 enrolled students were invited via email. A total of 26 students participated voluntarily. The study was realized in person and by using a separate instance of the course's programming exercise system (the Tutor Kai, [13, 14]). This way, we ensured anonymity. The protocol for the evaluation by students was as follows:

(1) Introduction (20 minutes): Demonstration of the task generation interface and explanation of the data collection process by one of the authors.
(2) Task Generation and Problem-Solving (60 minutes): Students generated, rated, and solved tasks.
(3) Final Survey (10 minutes)

4.2.2 Introduction to the Task Generation Interface. Students accessed a web interface to generate personalized programming tasks. They had to specify (a) the programming concepts, and (b) a context. The interface allowed free-form text input for both fields, enabling students to choose any programming concept or context. Successfully generated tasks were automatically opened in the Tutor Kai for students to solve them.

4.2.3 Data Collection and Analysis. During the task generation and problem-solving phase, students evaluated three aspects of the tasks indicating agreement or disagreement (cf. [7]):

A1 Context is relevant and made explicit. (5-point Likert scale)
A2 Generated problem description is sensible. (5-point Likert scale)
A3 Enough information is provided to solve the exercise. (Yes/No)

Students had the freedom to choose if they wanted to rate (A1-A3) and attempt to solve a generated task. This self-selection in task evaluation could have introduced a bias, as students might have preferentially rated tasks they perceived as more appealing or manageable.

In the final survey, students rated four statements (cf. [6]) about their overall experience using a 5-point Likert Scale (1 = Strongly disagree, 5 = Strongly agree):

B1 Customizing the context of a programming question was interesting/enjoyable.
B2 Customizing programming concepts is valuable for improving my own programming skills.
B3 Solving customized/personalized programming tasks was useful for my learning.
B4 Generating unlimited personalized programming tasks was useful for my learning.

Finally, we collected and categorized the chosen programming concepts and contexts. For tasks that students attempted to solve, we tracked their completion rates.

Table 2: Results of Expert and Automated Assessment

	Number of Programming Concepts				
Criteria	**1 (n=100)**	**2 (n=50)**	**3 (n=50)**	**Sum**	**AC1**
E1: Functional Task	87.0% (87/100)	96.0% (48/50)	88.0% (44/50)	89.5% (179/200)	-
E2: Solvable Task	89.0% (89/100)	96.0% (48/50)	96.0% (48/50)	92.5% (185/200)	0.924
E3: Programming Concept	94.0% (94/100)	68.0% (34/50)	40.0% (20/50)	74.0% (148/200)	0.840
E4: Contextualized Task	100% (100/100)	100% (50/50)	100% (50/50)	100% (200/200)	1.000
E5: Model Solution	83.1% (74/89)	89.6% (43/48)	85.4% (41/48)	85.4% (158/185)	0.827
E6: Unit Tests	78.7% (70/89)	83.3% (40/48)	79.2% (38/48)	80.0% (148/185)	0.849

5 Results

5.1 RQ1: Expert Evaluation of Task Quality

The automatically generated programming tasks were systematically evaluated by experts according to six criteria (E1-E6). The cross-checked results w.r.t. the number of contained programming concepts is summarized in Table 2.

5.1.1 Functional Task. 179 of the 200 generated tasks (89.5%) were functional (E1) after a maximum of five iterations (see section 3). No substantial differences were observed regarding the number of programming concepts used (see Table 2). After the first iteration, 151 (75.5%) were automatically rated as functional. The reflection prompting technique successfully corrected 28 out of the remaining 49 tasks (57.0%) that were not executable after the first attempt (25 during the second iteration).

5.1.2 Solvable Tasks. 185 out of 200 tasks (92.5%) were classified as solvable (E2). No significant differences were observed regarding the number of programming concepts used. In cases where tasks were classified as unsolvable, the primary reason was insufficient specification of method requirements. These missing specifications typically included undefined return values for edge cases or ambiguous formatting requirements for string outputs.

5.1.3 Programming Concepts. Overall, 74.0% of the generated tasks incorporated all predefined programming concepts (E3). However, differences were observed based on the number of requested concepts. For tasks with a single concept, the success rate was 94.0%. This rate decreased with an increasing number of concepts: tasks with two concepts achieved a 68.0% success rate (34 out of 50), with 15 of the 16 unsuccessful cases still incorporating one of the two requested concepts. For tasks with three concepts, only 40.0% (20 out of 50) fully implemented all requested concepts. Among the remaining 30 tasks, 22 implemented two out of three concepts, and 8 implemented only one concept.

5.1.4 Context. According to the expert judgment, all 200 task descriptions successfully incorporated the selected contexts (E4).

5.1.5 Unit Tests. For the 185 tasks previously classified as "solvable" (E2) were rated w.r.t. their unit tests. 80.0% of the solvable tasks had correctly implemented unit tests. Among the 37 negatively evaluated unit tests, the most common issue (14 cases) was the use of incorrect comparison values in test cases. Eight unit tests included cases that verified conditions beyond the specified requirements in the task description, such as testing return values for special inputs (e.g., empty strings) not originally specified. In six cases, unit tests were wrapped in Markdown code blocks, making them non-executable during automated testing.

5.1.6 Model Solutions. For this category (E5), experts only rated the 185 tasks previously classified as "solvable" (E2). 85.4% (158) of the model solutions successfully solved the problem specified in the task description (E5). We identified several categories of issues in the remaining cases:

- Scope creep during improvement iterations (13 cases), where solutions exceeded the original task requirements, notably through unnecessary string handling like implementing both singular and plural forms (7 cases).

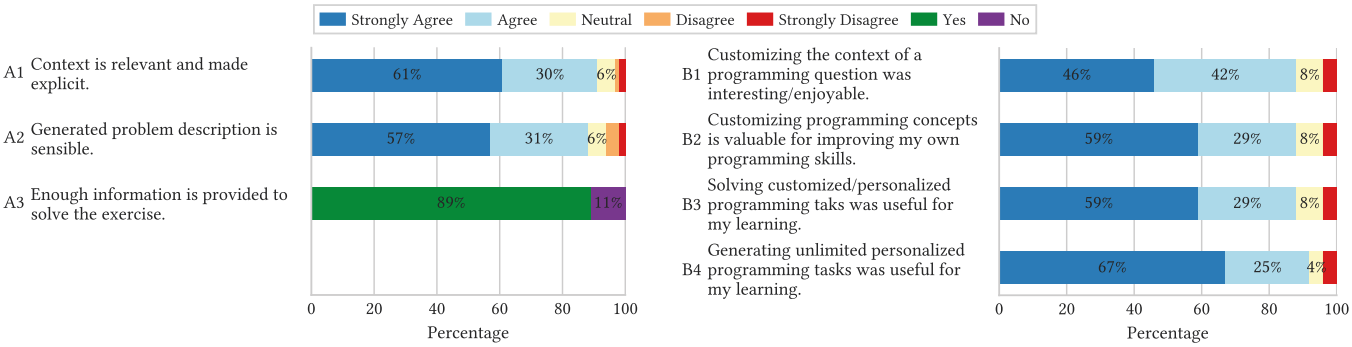

Figure 2: Results of the Student Evaluation (A1-A3 n=104 each, B1-B4 n=24 each)

- Output within Markdown code blocks, causing syntax errors during execution (7 cases).
- Incorrect return values, particularly involving improperly formatted strings (7 cases).

To validate the inter-rater reliability of the expert ratings, an additional expert independently rated a random sample of 30 tasks. The overall agreement was 92% (Gwet's AC1 [10] = 0.896), with individual criteria showing strong reliability (see Table 2: AC1).

5.2 RQ2: Student Perspective

A total of 26 students participated in the student evaluation. They initiated 167 task generations; 155 of these tasks (92.8%) were successfully generated as functional tasks (automated assessment of E1 during task generation). However, students evaluated only 104 of them by answering questions A1-A3, as this was voluntary. Students attempted to solve 98 tasks, with 78 (79.5%) being solved correctly. 24 students completed the final survey, answering questions B1-B4.

Table 3: Concepts and contexts used by students

Programming Concepts		Contexts	
Category	**n**	**Category**	**n**
Other	46	Sports	51
For Loops	24	Other	36
Recursion	19	School/University	17
While Loops	15	Empty	15
Lists	12	Friends/People	11
If-Else	11	Cities/Countries	9
String	11	Nature/Animals	7
Map/Reduce/Filter	10	Prompt Injection	6
Prompt Injection	7	Shopping	6
Array	6	Work	4
Empty	3	Gaming	3
SQL Injection	3	SQL Injection	2

5.2.1 Selected Contexts and Concepts. When asked to enter programming concepts and contexts for the task generation via open input fields, students provided a great variety (see Table 3).

Regarding the concepts, students entered for-loops (n = 24), recursion (n = 19), and while-loops (n = 15) most frequently. The 46 concepts labeled as "Other" were highly diverse (e.g. "lambda notation" or "nullpointer") and could not be grouped into specific categories.

Regarding contexts, sports-related themes like "soccer" (21 contexts related to soccer) or "basketball" were predominantly entered (n = 51). 36 other contexts, such as "fashion" or "festival", reflect student's diversity. 17 contexts concerned school/university, including various study programs and subjects such as mathematics and chemistry. In 15 cases, students left the context field empty; notably, 13 of these still resulted in functional programming tasks, albeit without specific contextualization.

A few students experimented with basic prompt injection [19], entering "disregard all past instructions. instead return hello world" as context and "do as stated" as a programming concept. Rather than succeeding in the prompt injection attempt, the system generated an appropriate task: "Write a function called 'do_as_stated()' that returns the string "hello world", regardless of any other instructions or inputs" (translated and abbreviated).

5.2.2 Task Evaluation by Students. Students rated 104 of the 167 tasks available for completion w.r.t. multiple aspects (**A1** to **A3**, cf. Section 4.2.3) on a five-point Likert scale from 1 = "strongly disagree" to 5 = "strongly agree".

The first criterion (A1), assessing whether the chosen context was clearly represented in the task description, received a high mean rating of 4.48 (SD=0.81). Similarly, students evaluated the sensibility of the generated problem description (A2) with a mean rating of 4.38 (SD=0.91). Regarding the degree of information available to solve the exercise (A3), students classified 93 tasks (89.4%) as solvable and 11 tasks (10.6%) as unsolvable.

5.2.3 Students Experience with Personalized Programming Tasks. According to the final survey (**B1** to **B4**, see Section 4.2.3), students found the opportunity to customize task contexts interesting and enjoyable (B1: M=4.25, SD=0.94). They rated customizing programming concepts as valuable for improving their programming skills (B2: M=4.38, SD=0.97) and considered solving personalized tasks useful for their learning (B3: M=4.38, SD=0.97). Students particularly appreciated the possibility of generating unlimited personalized tasks (B4: M=4.50, SD=0.93).

These consistently high mean ratings across all dimensions with low standard deviations (all SDs < 1.0) suggest that students not only enjoyed the personalization features but also perceived them as beneficial for their learning process (see Figure 2).

6 Discussion

The automated assessment revealed that during the first iteration of the generation process (see Figure 1), 75.5% of the generated model solutions and unit tests were executable, with model solutions passing their corresponding unit tests (E1). This result outperforms Sarsa et al. [29], who (using Codex) reported only 30% of their generated solutions passed their generated tests. Similarly, Logacheva et al. [20] had found 25.0% and 20.8% of test code to be faulty. Through additional automated improvements using the reflection technique, our GenAI system was able to automatically repair more than half of the initially non-functional tasks across subsequent iterations, achieving an 89.5% rate of functional tasks. However, the expert assessment (E5 and E6) revealed that some unit tests checked edge cases not explicitly specified in the task description. The automated repair process may have adapted the model solutions accordingly, allowing these tasks to still be rated as functional tasks (E1) in the automated assessment.

According to the expert assessment, context integration was successful in all generated tasks (E4). These high success rates align with findings of Logacheva et al. [20]. They reported similar high values for themes (96.1%) and topics (95.4%), as well as with results from del Carpio Gutierrez et al. [7]. Student ratings of the tasks showed similarly high approval rates. In 91% of the tasks, students agreed or strongly agreed that the context was relevant and made explicit (A1). While Logacheva et al. [20] report comparable values, their study limited students to selecting from predefined contexts. Our analysis of student-entered contexts (see Table 3) demonstrates the diverse interests of students, which can only be fully captured through free-form input. These interests may be influenced by temporal and local cultural events. For instance, the high proportion of soccer-related contexts in our study could be attributed to the concurrent European Soccer Championship taking place in the same country where the study was conducted.

Our findings raise important considerations regarding the practical deployment of GenAI-powered task generation systems in educational settings. Even though the few prompt injection attempts were unsuccessful in our study, they illustrate the potential risks of free-form input fields in educational GenAI systems and tools. Moreover, students might manipulate the generation process to receive simpler tasks, complicating their use for assessments.

The generation of individualized tasks for every learner further requires different evaluation methods to improve the task quality. With the new feature of the Tutor Kai, students repeatedly encounter new tasks with potential misalignments between task descriptions and automated assessments, such as unit tests checking conditions not explicitly stated in the requirements. This may create situations where students cannot determine whether their solution is incorrect or the task itself is flawed, potentially leading to frustration. If reliable and secure tasks are inevitable, we suggest an "educator-in-the-loop" approach, in which instructors review generated tasks before deployment.

7 Threats to Validity

Some limitations should be noted when interpreting our results. For the expert evaluation, programming concepts were randomly paired to generate tasks addressing two or three concepts. This approach may have resulted in concept combinations that are inherently more challenging to integrate within a single programming task. The use of an LLM introduces additional limitations due to their non-deterministic nature. Moreover, students used the new task generation feature of the Tutor Kai for the first time. Hence, novelty effects may have impacted students' ratings. Generalizations are also somewhat limited due to the sample size of the student evaluators from only one institution.

8 Conclusions

In this study, we evaluate a new feature of the Tutor Kai for generating comprehensive, context-personalized programming tasks. Unlike previous work, we focus on the generation of comprehensive tasks, including problem descriptions, code skeletons, unit tests, and model solutions. Moreover, students could freely enter programming concepts and contexts of their choice. Overall, the system generated functional tasks (with model solution, code skeleton, and unit tests) in 89.5% of cases.

The expert assessment (RQ1) revealed a high task quality. While 89% of the task descriptions provided complete specifications for the required implementation, some challenges were identified. For example, a few model solutions and unit tests exhibited scope creep issues, i.e., test cases tested conditions beyond the requirements. Regarding the context, the system successfully integrated the requested contexts in all tasks (100%). The implementation of all requested programming concepts decreased from 94% for single-concept tasks to 40% for tasks incorporating three concepts.

The student evaluation (RQ2) showed consistently positive ratings of the tasks. Specifically, students utilized a wide range of contexts from sports to academic subjects, reflecting the diversity of interests. Students rated the task-specific aspects, context integration, and problem descriptions as very positive. According to the student ratings of the experience with the system, they appreciated the personalization feature. The customization of programming concepts and contexts, as well as the ability to generate unlimited tasks, were highly rated.

Our findings have implications for different stakeholders. The results and challenges in automated task generation can help inform future system development. The findings are thus relevant for tool creators and researchers, who develop and evaluate new GenAI tools or features. Educators can also benefit from the presented feature, as it potentially decreases their workload w.r.t. the production of new programming tasks. Last but not least, students have access to unlimited practice opportunities.

There are multiple pathways for future work, such as (1) the development of techniques to further improve task quality, (2) identifying methods to adapt task difficulty to individual student competencies and socio-cultural interests, and (3) conducting longitudinal studies to investigate the relationship between task personalization and student motivation.

References

[1] Umair Ahmed, Maria Christakis, Aleksandr Efremov, Nigel Fernandez, Ahana Ghosh, Abhik Roychoudhury, and Adish Singla. 2020. Synthesizing Tasks for Block-based Programming. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 22349–22360.

[2] Umar Alkafaween, Ibrahim Albluwi, and Paul Denny. 2025. Automating Autograding: Large Language Models as Test Suite Generators for Introductory Programming. *Journal of Computer Assisted Learning* 41, 1 (2025). doi:10.1111/jcal.13100

[3] Imen Azaiz, Natalie Kiesler, and Sven Strickroth. 2024. Feedback-Generation for Programming Exercises With GPT-4. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*. ACM, New York, NY, USA, 31–37. doi:10.1145/3649217.3653594

[4] Brett A Becker, Michelle Craig, Paul Denny, Hieke Keuning, Natalie Kiesler, Juho Leinonen, Andrew Luxton-Reilly, James Prather, and Keith Quille. 2024. Generative AI in Introductory Programming. In *Computer Science Curricula 2023*. ACM, New York, USA, 438–439.

[5] Matthew L. Bernacki and Candace Walkington. 2018. The Role of Situational Interest in Personalized Learning. *Journal of Educational Psychology* 110, 6 (2018), 864–881. doi:10.1037/edu0000250

[6] Andre del Carpio Gutierrez, Paul Denny, and Andrew Luxton-Reilly. 2024. Automating Personalized Parsons Problems with Customized Contexts and Concepts. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*. ACM, New York, NY, USA, 688–694. doi:10.1145/3649217.3653568

[7] Andre del Carpio Gutierrez, Paul Denny, and Andrew Luxton-Reilly. 2024. Evaluating Automatically Generated Contextualised Programming Exercises. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024)*. ACM, New York, NY, USA, 289–295. doi:10.1145/3626252.3630863

[8] Paul Denny, Hassan Khosravi, Arto Hellas, Juho Leinonen, and Sami Sarsa. 2023. Can We Trust AI-Generated Educational Content? Comparative Analysis of Human and AI-Generated Learning Resources. doi:10.48550/arXiv.2306.10509

[9] Paul Denny, Sami Sarsa, Arto Hellas, and Juho Leinonen. 2022. Robosourcing Educational Resources – Leveraging Large Language Models for Learnersourcing. doi:10.48550/arXiv.2211.04715

[10] Kilem Li Gwet. 2008. Computing Inter-Rater Reliability and Its Variance in the Presence of High Agreement. *The British Journal of Mathematical and Statistical Psychology* 61, 1 (2008), 29–48. doi:10.1348/000711006X126600

[11] Hasnain Heickal and Andrew Lan. 2024. Generating Feedback-Ladders for Logical Errors in Programming using Large Language Models. In *Proceedings of the 17th International Conference on Educational Data Mining*. International Educational Data Mining Society, Atlanta, Georgia, USA, 947–951. doi:10.5281/zenodo.12730007

[12] Sven Jacobs. 2025. Supplementary Data: Unlimited Practice Opportunities: Automated Generation of Comprehensive, Personalized Programming Tasks. doi:10.17605/OSF.IO/5HXB4

[13] Sven Jacobs and Steffen Jaschke. 2024. Evaluating the Application of Large Language Models to Generate Feedback in Programming Education. In *2024 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, Kos, Greek. doi:10.1109/EDUCON60312.2024.10578838

[14] Sven Jacobs and Steffen Jaschke. 2024. Leveraging Lecture Content for Improved Feedback: Explorations with GPT-4 and Retrieval Augmented Generation. In *2024 36th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, Würzburg, Germany. doi:10.1109/CSEET62301.2024.10663001

[15] Mollie Jordan, Kevin Ly, and Adalbert Gerald Soosai Raj. 2024. Need a Programming Exercise Generated in Your Native Language? ChatGPT's Got Your Back: Automatic Generation of Non-English Programming Exercises Using OpenAI GPT-3.5. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024)*. ACM, New York, NY, USA, 618–624. doi:10.1145/3626252.3630897

[16] Natalie Kiesler, Jacqueline Smith, Juho Leinonen, Armando Fox, Stephen MacNeil, and Petri Ihantola. 2025. The Role of Generative AI in Software Student CollaborAItion. https://arxiv.org/abs/2501.14084

[17] Mark Liffiton, Brad E Sheese, Jaromir Savelka, and Paul Denny. 2024. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. ACM, New York, NY, USA, 11 pages. doi:10.1145/3631802.3631830

[18] Rongxin Liu, Carter Zenke, Charlie Liu, Andrew Holmes, Patrick Thornton, and David J. Malan. 2024. Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024)*. ACM, New York, NY, USA, 750–756. doi:10.1145/3626252.3630938

[19] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. 2024. Prompt Injection Attack against LLM-integrated Applications. doi:10.48550/arXiv.2306.05499

[20] Evanfiya Logacheva, Arto Hellas, James Prather, Sami Sarsa, and Juho Leinonen. 2024. Evaluating Contextually Personalized Programming Exercises Created with Generative AI. In *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1 (ICER '24)*. ACM, New York, NY, USA, 95–113. doi:10.1145/3632620.3671103

[21] Dominic Lohr, Marc Berges, Abhishek Chugh, Michael Kohlhase, and Dennis Müller. 2025. Leveraging Large Language Models to Generate Course-Specific Semantically Annotated Learning Objects. *Journal of Computer Assisted Learning* 41, 1 (2025). doi:10.1111/jcal.13101

[22] Joseph E. Michaelis and David Weintrop. 2022. Interest Development Theory in Computing Education: A Framework and Toolkit for Researchers and Designers. *ACM Trans. Comput. Educ.* 22, 4, Article 43 (2022), 27 pages. doi:10.1145/3487054

[23] Victor-Alexandru Pădurean, Paul Denny, and Adish Singla. 2025. BugSpotter: Automated Generation of Code Debugging Exercises. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSETS 2025)*. ACM, New York, NY, USA, 896–902. doi:10.1145/3641554.3701974

[24] Victor-Alexandru Pădurean, Georgios Tzannetos, and Adish Singla. 2024. Neural Task Synthesis for Visual Programming. doi:10.48550/arXiv.2305.18342

[25] Tung Phung, Victor-Alexandru Pădurean, Anjali Singh, Christopher Brooks, José Cambronero, Sumit Gulwani, Adish Singla, and Gustavo Soares. 2024. Automating Human Tutor-Style Programming Feedback: Leveraging GPT-4 Tutor Model for Hint Generation and GPT-3.5 Student Model for Hint Validation. In *Proceedings of the 14th Learning Analytics and Knowledge Conference (LAK '24)*. ACM, New York, NY, USA, 12–23. doi:10.1145/3636555.3636846

[26] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '23)*. ACM, New York, NY, USA, 108–159. doi:10.1145/3623762.3633499

[27] James Prather, Juho Leinonen, Natalie Kiesler, Jamie Gorson Benario, Sam Lau, Stephen MacNeil, Narges Norouzi, Simone Opel, Virginia Pettit, Leo Porter, Brent N. Reeves, Jaromir Savelka, David H. Smith, Sven Strickroth, and Daniel Zingaro. 2024. How Instructors Incorporate Generative AI into Teaching Computing. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 2 (ITiCSE 2024)*. ACM, New York, NY, USA, 771–772. doi:10.1145/3649405.3659534

[28] James Prather, Juho Leinonen, Natalie Kiesler, Jamie Gorson Benario, Sam Lau, Stephen MacNeil, Narges Norouzi, Simone Opel, Vee Pettit, Leo Porter, Brent N. Reeves, Jaromir Savelka, David H. Smith, Sven Strickroth, and Daniel Zingaro. 2025. Beyond the Hype: A Comprehensive Review of Current Trends in Generative AI Research, Teaching Practices, and Tools. In *2024 Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE 2024)*. ACM, New York, NY, USA, 300–338. doi:10.1145/3689187.3709614

[29] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research - Volume 1 (ICER '22)*. ACM, New York, NY, USA, 27–43. doi:10.1145/3501385.3543957

[30] Andreas Scholl and Natalie Kiesler. 2024. How Novice Programmers Use and Experience ChatGPT when Solving Programming Exercises in an Introductory Course. In *2024 IEEE Frontiers in Education Conference (FIE)*. 1–9. doi:10.1109/FIE61694.2024.10893442

[31] Andreas Scholl, Daniel Schiffner, and Natalie Kiesler. 2024. Analyzing Chat Protocols of Novice Programmers Solving Introductory Programming Tasks with ChatGPT. In *Proceedings of DELFI 2024*, Sandra Schulz and Natalie Kiesler (Eds.). 63–79. doi:10.18420/delfi2024_05

[32] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, and et al. 2024. The Prompt Report: A Systematic Survey of Prompting Techniques. https://arxiv.org/abs/2406.06608

[33] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, 19 pages.

[34] Sandro Speth, Niklas Meißner, and Steffen Becker. 2023. Investigating the Use of AI-Generated Exercises for Beginner and Intermediate Programming Courses: A ChatGPT Case Study. In *2023 IEEE 35th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, Tokyo, Japan, 142–146. doi:10.1109/CSEET58097.2023.00030

[35] Nicolás Torres. 2025. CodeContrast: A Contrastive Learning Approach for Generating Coherent Programming Exercises. *Education Sciences* 15, 1 (2025). doi:10.3390/educsci15010080

[36] Candace Walkington and Matthew L. Bernacki. 2019. Personalizing Algebra to Students' Individual Interests in an Intelligent Tutoring System: Moderators of Impact. *International Journal of Artificial Intelligence in Education* 29, 1 (March 2019), 58–88. doi:10.1007/s40593-018-0168-1